FACULDADE IBRATEC

Programação Orientada a Objetos em Java

Prof. Msc Marcos de Souza

A Classe String

- São seqüências de caracteres
- Não há um tipo primitivo para Strings em Java
- Em Java, Strings são objetos

```
String mensagem = “Operação concluída com sucesso”;
```

Aqui Java cria um novo objeto do tipo String e o armazena na variável mensagem

A Classe String

- Operador + é usado para concatenação

```
String nome = "George";
String sobrenome = "Bush";
String nomeCompleto = nome + " " + sobrenome;

int anos = 10;
double rendimento = 1270.49;

String s = "Em " + anos + " anos o " +
 "rendimento será de " + rendimento;

System.out.println(s);
```

A conversão para String é feita automaticamente

A Classe String

- Para testar se dois Strings são iguais, não deve ser usado ==
- Deve-se usar o método equals():
  - Método alternativo: equalsIgnoreCase()

```
s1 == s2
```

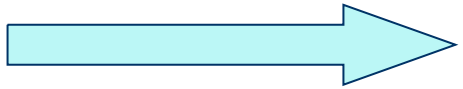
```
s1.equals(s2)
```

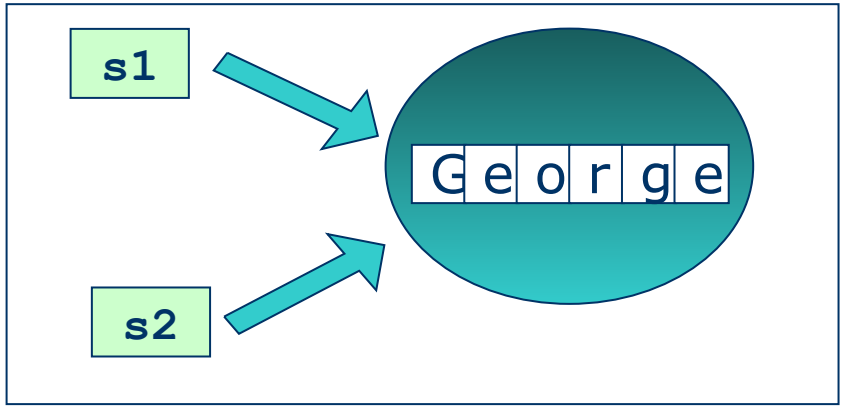
A Classe String

```
//Cria dois novos Strings
String s1 = "George";
String s2 = "George";
// Neste momento, s1==s2 é verdadeiro!

s1 = s1 + " Bush"; // Cria um novo string e o atribui para s1
s2 = s2 + " Bush"; // Cria um novo string e o atribui para s2
// Neste momento, s1==s2 não é verdadeiro!


if (s1 == s2) {
  System.out.println("s1 e s2 sao os mesmos objetos.");
} else {
  System.out.println("s1 e s2 NAO sao os mesmos objetos.");
}
if (s1.equals(s2)) {
  System.out.println("s1 e s2 sao iguais.");
} else {
  System.out.println("s1 e s2 NAO sao iguais.");
}
```

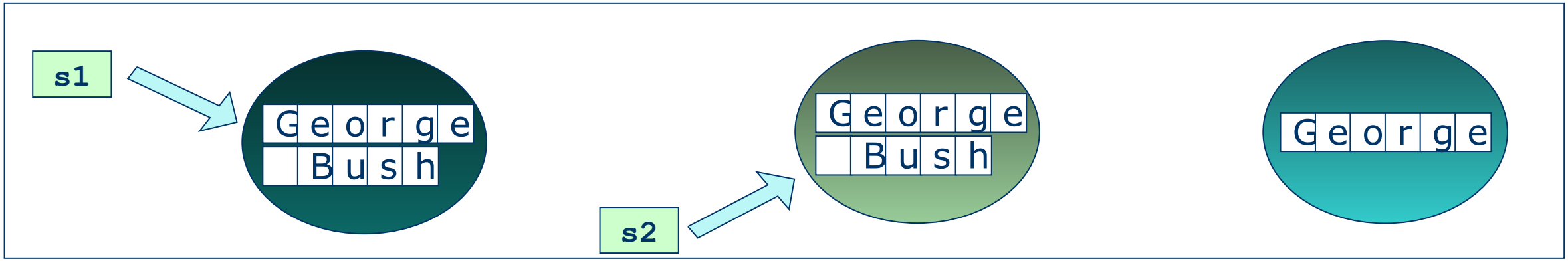
A Classe String

```
String s1 = "George";
String s2 = "George";

s1 = s1 + " Bush";
s2 = s2 + " Bush";
```





Ao manipular um String,
um novo String é gerado!



A Classe String

- boolean equals(uma String)
- boolean equalsIgnoreCase(uma String)
- int length()

```
String a = "Sharon Stone";
String b = "sharon stone";
int comprimento = a.length();
boolean resposta1 = a.equals(b);
boolean resposta2 = a.equalsIgnoreCase(b);
boolean resposta3 = b.equalsIgnoreCase(a);
boolean resposta4 = (comprimento > 5);
```

Qual o valor das respostas?

A Classe String

- umaString.toLowerCase()
- umaString.toUpperCase()
- umaString.trim()

```
String x = " Bom Dia! ";
String y = x.toUpperCase();
String z = x.toLowerCase();
String w = x.trim();
System.out.println(y);
System.out.println(z);
System.out.println(w);
```

```
 BOM DIA!
 bom dia!
Bom Dia!
```

A Classe String

- int indexOf(uma String)
- umaString.substring(int indiceInicio, int indiceFinal)
- umaString.substring(int indiceInicio)
- char charAt(int indice)

```
String x = "Pernambuco";
String y = x.substring(0,5);
String z = x.substring(6);
int indice = x.indexOf("na");
char letra = x.charAt(5);
System.out.println(x); System.out.println(y);
System.out.println(z); System.out.println(indice);
System.out.println(letra);
```

Qual é a saída?

A Classe String

```
public class TV {
    private String marca;
    private String modelo;
            private int tamanho;
...

public String toString() {
    return "TV" +
                        "\nMarca: " + this.marca +
                "\nModelo: " + this.modelo +
                        "\nTamanho: " + this.tamanho;
}

public static void main(String args[]) {
    TV tv1 = new TV("Marca1", "SDX-22", 29);
    System.out.println(tv1.toString());
}
```

A Classe String

- Em Java existem classes que encapsulam tipos primitivos (*wrappers*)
  - Float, Integer, Log, Character, Double, Float, Boolean

- Estas classes contém métodos úteis para transformar uma String em um tipo primitivo.
  - Integer.parseInt(s)
  - Double.parseDouble(s)
  - Boolean.parseBoolean(s)

- Para transformar de tipos primitivos para String usa-se:
  - String.valueOf(x)

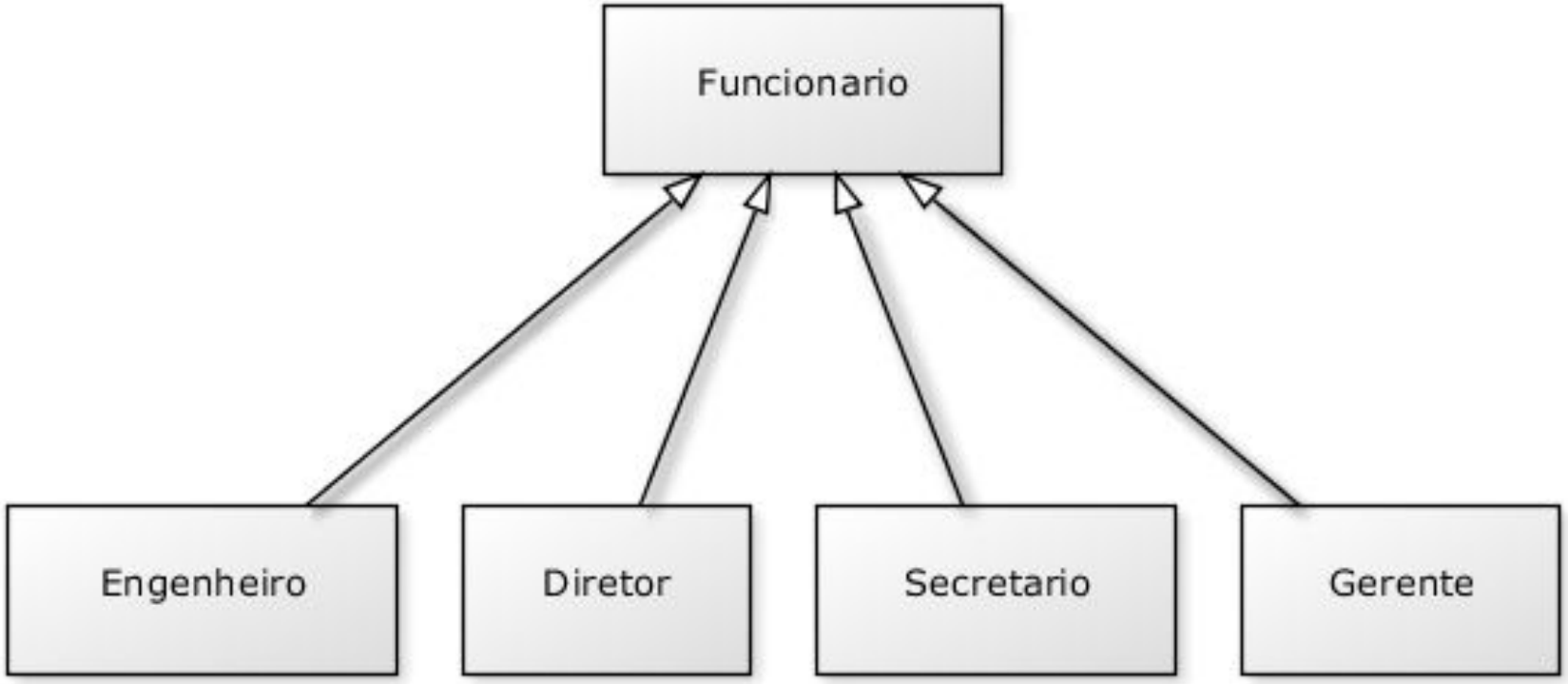
Herança

FACULDADE IBRATEC

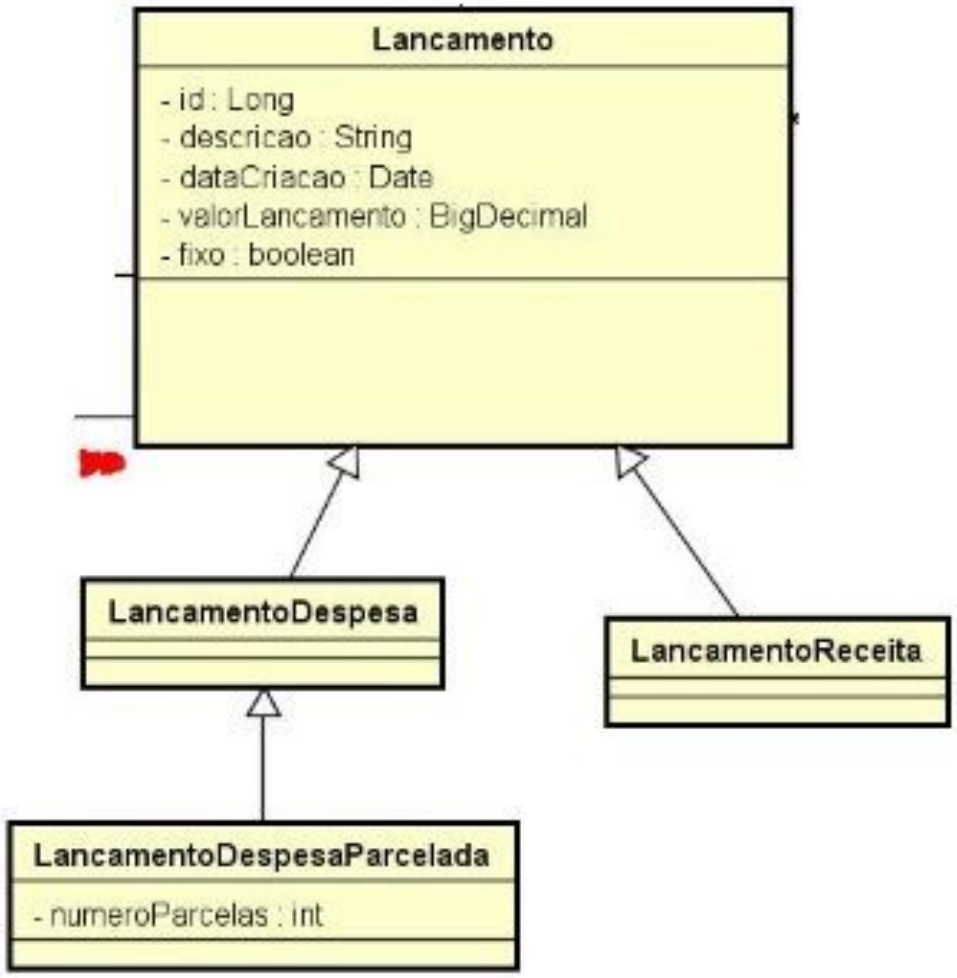
Herança

- **Alguns atributos/comportamentos dos objetos podem ser compartilhados**
- **Como podemos evitar a redundância de código?**
- **Concentração de determinadas regras a uma classe mais genérica**

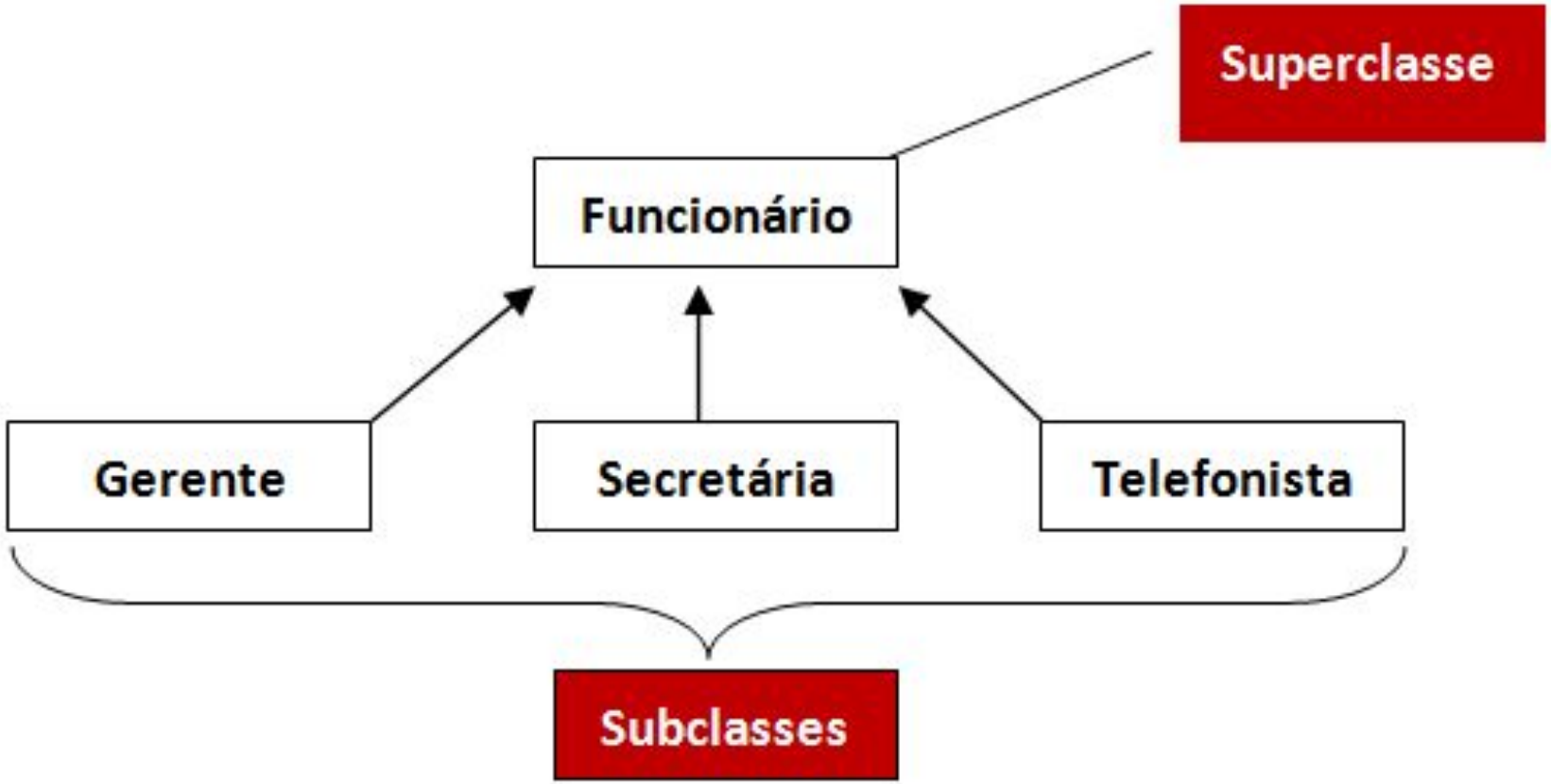
Herança



Herança


Lancamento
- id : Long
- descricao : String
- dataCriacao : Date
- valorLancamento : BigDecimal
- fixo : boolean
LancamentoDespesa
LancamentoReceita
LancamentoDespesaParcelada
- numeroParcelas : int

Herança


Superclasse
Funcionário
Gerente
Secretária
Telefonista
Subclasses

Herança

Item.java

```java
public class Item
{
    private boolean titulo;
    private boolean duracaoTotal;
    private boolean comentario;

    // MÉTODOS OMITIDOS
```

CD.java

```java
public class CD extends Item
{
    private boolean artista;
    private boolean qtdFaixas;

    //MÉTODOS OMITIDOS
```

Para criar uma **subclasse**, basta adicionar a palavra **extends** e o nome da **superclasse**.

Herança

```
class Funcionario{
    String nome;
    double salario;

    Funcionario(String nome, double salario){
        this.nome = nome;
    }
}
```

```
class Secretaria extends Funcionario{
    String ramal;

    Secretaria(String nome, double salario, String ramal){
        super(nome, salario);
        this.ramal = ramal;
    }
}
```

```
class Gerente extends Funcionario{
    int sala;

    Gerente(String nome, double salario, int sala){
        super(nome, salario);
        this.sala = sala;
    }
}
```

Exceções

Exceções

FACULDADE IBRATEC

Exceções

- Sistemas robustos devem:
  - Fornecer formas eficientes para recuperação de falhas.
  - Fornecer mensagens apropriadas de erros.
  - Fazer validação dos dados.
  - Garantir a consistência das operações:
    - Retornar a um estado seguro e estável permitindo ao usuário executar outros comandos.
    - Permitir ao usuário salvar todo o seu trabalho.

Exceções

- Como evitar débitos que deixam o saldo negativo?

```
pubic class Conta {

  public void debitar(double valor) {
    saldo = saldo - valor;
  }
}
```

- Desconsiderar operação
- Mostrar mensagem de erro
- Retornar código de erro

Exceções

```
public class Conta {

  public void debitar(double valor) {
    if (valor <= saldo) {
      saldo = saldo - valor;
    }
  }
}
```

- **Problemas:**
  - quem solicita a operação não tem como saber se ela foi realizada ou não;
  - nenhuma informação é dada ao usuário do sistema

Exceções

```
public class Conta {
  public void debitar(double valor) {
    if (valor <= saldo){
      saldo = saldo - valor;
    }else {
      System.out.println("Saldo Insuficiente!");
    }
  }
}
```

- Problema:
  - informação é dada ao usuário do sistema, mas nenhuma sinalização de erro é fornecida para quem invocou debitar.

Exceções

```
public class Conta {

  public boolean debitar(double valor) {
    boolean r = false;
    if (valor <= saldo) {
      saldo = saldo - valor;
      r = true;
    }

    return r;
  }
}
```

Exceções

- Problemas:
  - dificulta a definição e o uso do método;
  - métodos que invocam debitar têm que testar o resultado retornado para decidir o que deve ser feito;
  - A dificuldade é maior para métodos que retornam valores:
    - Se debitar já retornasse um outro valor qualquer? O que teria que ser feito?

Exceções

- Exceções são o mecanismo suportado por Java para tratamento de erros ou situações indesejadas.

- Exceções são objetos que encapsulam informações relevantes sobre o erro ocorrido.

- Exceções em Java são indicadas nos lugares em que elas PODEM ocorrer.

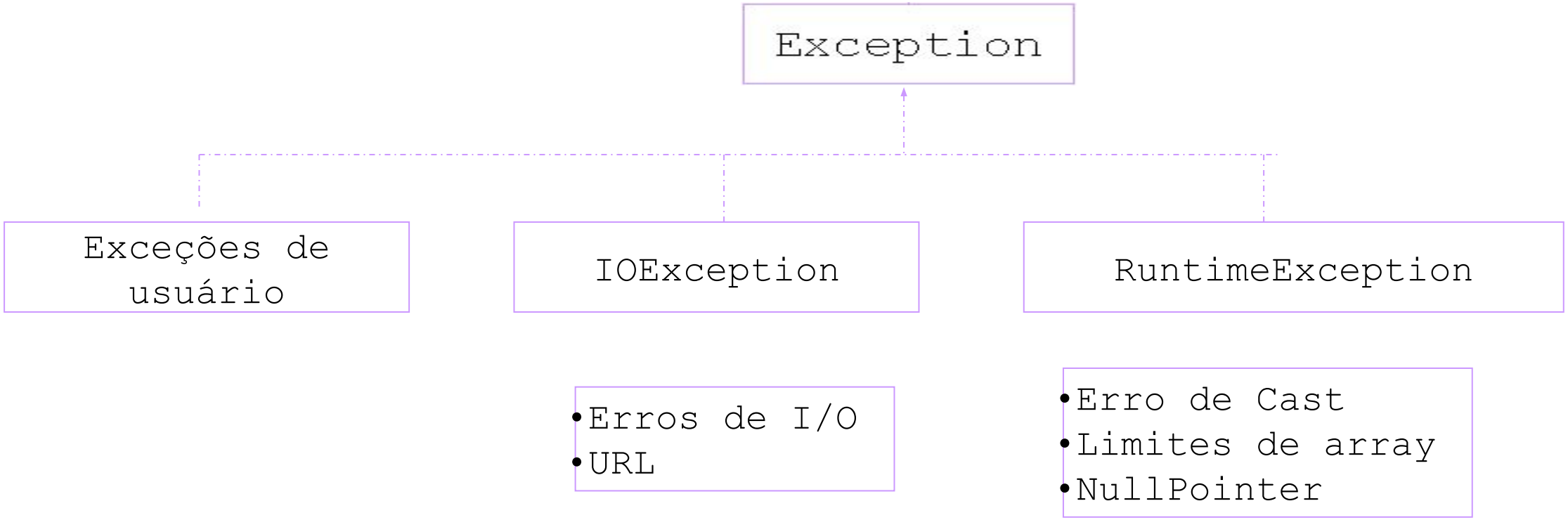
Exceções

- Exceções

- Nos lugares em que se deseja tomar alguma providência quando uma exceção ocorre, usa-se tratamento de exceções.

- Exceções em Java são
  - Declaradas - indicadas onde podem ocorrer;
  - Lançadas - passadas para alguém
    - Criadas explicitamente
    - Repassadas
  - Tratadas - capturadas para se tomar providências.

Exceções


Exception
Exceções de usuário
IOException
RuntimeException
•Erros de I/O
•URL
•Erro de Cast
•Limites de array
•NullPointer

Exceções

- Criadas para representar os erros que podem acontecer no sistema:
  - Erros de negócio;
  - Erros de persistência

Exceções

```
public class SaldoInsuficenteException
   extends Exception {

  private String numero;

  public SaldoInsuficienteException(String numero) {
    super("Saldo insuficiente!");
    this.numero = numero;
  }
  public String getNumero() {
   return numero;
  }

}
```

Exceções

- Todo método que pode gerar uma situação de erro pode levantar uma exceção;
- Exceções são ***declaradas*** na assinatura dos métodos usando **throws;**
- Exceções são ***lançadas*** no corpo dos métodos usando o comando **throw:**

```
void metodo(...) throws Exc1, Exc2, ExcN {
  ...
  throw new Exc1(...);
}
```

Exceções

- Criação explícita de exceções

```
public class Conta{
  ...

 public void debitar(double valor)
    throws SaldoInsuficienteException {

  if (valor <= saldo) {
    saldo = saldo - valor;
  }else {
   throw new
   SaldoInsuficienteException(getNumero());
  }
}
```

Exceções

- O método debitar pode levantar uma exceção.
- Quando a situação anormal ocorre (valor <= saldo), o funcionamento do método pára e um novo objeto do tipo SaldoInsuficienteException é criado e passado para quem chamou debitar.
- Quem chamou debitar vai ter que tratar ou repassar a exceção.

Exceções

- Exceções repassadas
  - Recebidas automaticamente, mas repassadas para alguém:

```
public class Conta{
 ...
 public void transferir(Conta c, double v) throws
   SaldoInsuficienteException {

   this.debitar(v);
   c.creditar(v);
 }
}
```

Checked unchecked

- Unchecked Exceptions são exceções que NÃO precisam ser declaradas nos métodos em que elas podem acontecer.
  - Herdam de RuntimeException.

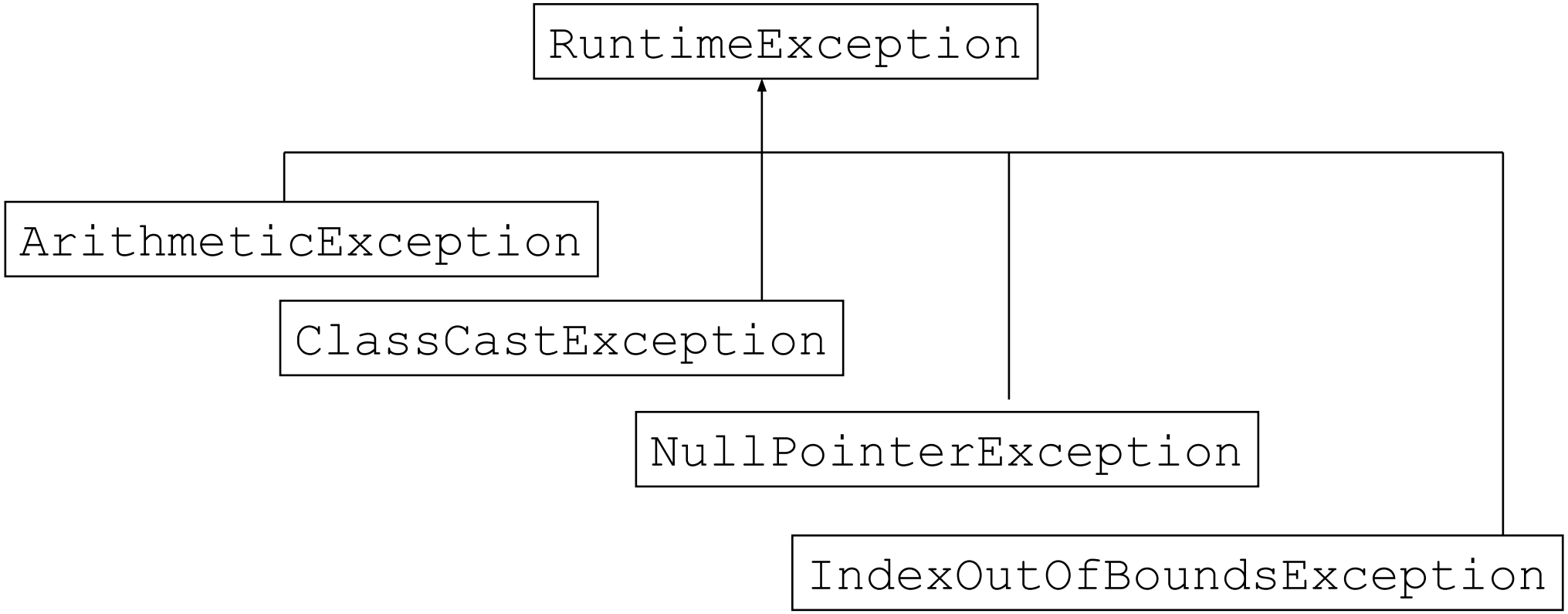
- Checked Exceptions são exceções que DEVEM ser declaradas nos métodos em que elas podem acontecer
  - Todas as outras exceções que não são subclasses de RuntimeException.

Exceções

```
//EXCEÇÃO NÃO PRECISA SER INDICADA
public void calculaPreco() throws RuntimeException{

   ...
}
```

```
//EXCEÇAO PRECISA SER INDICADA
public void debitar()
    throws SaldoInsuficienteException{

   ...
}
```

Exceções


RuntimeException
ArithmeticException
ClassCastException
NullPointerException
IndexOutOfBoundsException

Exceções

- As classes RepositorioContasArray e RepositorioClientesArray representam coleções de objetos que devem retornar erros (Exceções) nas seguintes situações:
  - Atualização de um objeto que não existe;
  - Remoção de um objeto que não existe;
  - Procura por um objeto que não existe;
  - ClienteInexistenteException;
  - ContaInexistenteException

Exceções

```
public class ContaInexistenteException
  extends Exception {

   public String numero;

  public ContaInexistenteException(String numero) {
    super("Conta" + numero + " não cadastrada");
    this.numero = numero;
  }
  public String getNumero() {
    return numero;
  }
}
```

Exceções

```
public class RepositorioContas {
 ...
 public Conta procurar(String numero)


 public void remover(String numero)


 public void atualizar(Conta conta)


 ...

}
```

Exceções

```
public class RepositorioContas {
 ...
 public Conta procurar(String numero)
    throws ContaInexistenteException

 public void remover(String numero)
       throws ContaInexistenteException

 public void atualizar(Conta conta)
       throws ContaInexistenteException

 ...

}
```

Exceções

```
public class RepositorioContasArray {
```

```
public class RepositorioContasArray {
 ...
 public Conta procurar(String numero)
   throws ContaInexistenteException{

  Conta c = null;
  if (existe(numero)) {
    int i = this.procurarIndice(numero);
    c = contas[i];
  } else {
    throw new
      ContaInexistenteException(numero);
  }
  return c;
 }
}
```

Exceções

```
public void atualizar(ContaAbstrata conta)
    throws ContaInexistenteException{

  int i = procurarIndice(conta.getNumero());
  if (i != -1) {
    contas[i] = conta;
  } else {
    throw
      new ContaInexistenteException(conta.getNumero());
  }
}
```

Exceções

```
public void remover(String numero)
  throws ContaInexistenteException{

  if (existe(num)) {
    int i = this.procurarIndice(numero);
    contas[i] = contas[indice - 1];
    contas[indice - 1] = null;
    indice = indice - 1;
  }else {
   throw
    new ContaInexistenteException(numero);
  }
}
```

Exceções

- O tratamento de exceções é utilizado quando se quer tomar alguma providência quando uma exceção acontece.
- A máquina virtual executa um código com maior atenção para capturar as exceções que podem acontecer.
- Quando a exceção acontece, o estado do sistema até o momento antes do acontecimento do erro é guardado.
- A exceção é passada para o fluxo de controle, que executa alguma ação.

Exceções

- Exceções são tratadas usando blocos: **try-catch [-finally]**

- O **finally** é opcional e sempre é executado, independente se aconteceu exceção ou não.

```
try {
 // chamada aos métodos que podem lançar
 // exceções
} catch (Exc1 ex) {
  // código para tratar um tipo de exceção
} finally {
  //codigo que vai ser sempre executado
}
```

Exceções

- Os catchs são analisados em ordem. Se a exceção é do tipo Exc1 o primeiro catch é executado e o fluxo segue após o último catch.

```
try {
 // chamada aos métodos que podem lançar
 //excecoes
} catch (Exc1 ex) {
  // código para tratar um tipo de exceção
} catch (ExcN ex) {
  // código para tratar exceção
} finally {
  //codigo que vai ser sempre executado
}
```

Exceções

```
try {
  contas.debitar("123-4",90.00);
  System.out.println("Débito efetuado");
}catch (SaldoInsuficienteException ex) {
 System.out.println(ex.getMessage());
 System.out.println(ex.getNumero());
}catch(ContaInexistenteException ex) {
  System.out.print(ex.getMessage());
  ex.printStackTrace();
}finally{
  System.out.print(“try executado”);
}
```

Exceções

- Evite deixar o bloco do *catch* vazio, mesmo que a exceção nunca venha a ocorrer

```
try {
  //codigo que levanta exceção
}catch (Exc1 ex) {
 // usar um System.out.print();
}
```

Exceções

- Procure tratar diferentes exceções num só bloco

```
try {
 //codigo que levanta Exc1
}catch (Exc1 ex) {...}

try {
 //codigo que levanta Exc2
}catch (Exc2 ex) {...}

try {
 //codigo que levanta Exc3
}catch (Exc3 ex) {...}
```

```
try {
 //codigo que levanta Exc1
 //codigo que levanta Exc2
 //codigo que levanta Exc3
}catch (Exc1 ex) {...}
}catch (Exc2 ex) {...}
}catch (Exc3 ex) {...}
```

Exceções

- Trate primeiro as exceções específicas

```
try {
 //codigo que levanta SaldoInsuficienteException
 //codigo que levanta ContaInexistenteException
}catch (SaldoInsuficienteException ex) {
 ...
}catch (ContaInexistenteException ex) {
 ...
}
}catch (Exception ex) {
 ...
```